Generación de árboles tropicales sintéticos a partir de modelos QSM reales: comparación de tres familias de generadores y bosque virtual con LiDAR y cámara de profundidad en Isaac Sim

Nelson Broyer-Paco Angel Marquez-Aguilar Li Shi Josine Pacifique

Código y datos: `https://github.com/Nosl3n/Synthetic_Tree_G`

30 de octubre de 2026

Resumen

Los modelos de percepción para entornos forestales necesitan muchos datos 3D etiquetados, y estos son caros de obtener. Este trabajo estudia cómo generar árboles sintéticos realistas a partir de 201 modelos estructurales cuantitativos (QSM) de árboles tropicales escaneados con láser terrestre en Barro Colorado Island (Panamá), y cómo medir su parecido con los reales. Implementamos ocho generadores que cubren las tres familias de la literatura: (i) métodos que aprenden de árboles reales (perturbación, perturbación alométrica, recombinación de ramas, híbrido y un generador procedural que sortea reglas de ramas reales); (ii) un generador basado solo en reglas botánicas (alometría, filotaxis, perfiles de adelgazamiento); y (iii) un algoritmo clásico de crecimiento (*space colonization*). Los evaluamos contra árboles reales no vistos mediante distancias de Wasserstein normalizadas sobre 22 rasgos de árbol y 10 de rama, un test de dos muestras por vecino más cercano (1-NNA), una métrica de novedad y reglas de validez botánica. La recombinación de ramas de la misma especie es el método más fiel (1-NNA = 56 % frente a 46 % entre reales), el generador procedural aprendido es el más novedoso entre los realistas, y su mezcla conserva la fidelidad de la recombinación aumentando la novedad de 0,73 a 1,19. Los generadores de reglas y de *space colonization* producen árboles casi perfectamente distinguibles (89 % y 98 %). Finalmente, construimos un bosque virtual de 24 árboles sintéticos en NVIDIA Isaac Sim, con ramas modeladas como tubos continuos, en dos versiones: solo árboles (v1) y un bosque ambientado con terreno, sotobosque, madera muerta, rocas y árboles exteriores (v2). Un LiDAR rotatorio Ouster OS1 de 128 canales y una cámara RGB-D simulados generan datos etiquetados por clase a 10 Hz, que se visualizan en tiempo real. En v2 el sotobosque supone el 21,7 % de los puntos LiDAR, una clase ausente en v1, mientras que la visibilidad de los troncos apenas cambia (−4 %).

Palabras clave: árboles sintéticos, QSM, LiDAR terrestre, modelado procedural inverso, evaluación de datos sintéticos, Isaac Sim, bosque tropical.

1. Introducción

La segmentación de nubes de puntos forestales (separar suelo, tronco, ramas y hojas, o individualizar árboles) es la base del inventario forestal automático y de la navegación de robots en cultivos y bosques. Las redes profundas que resuelven estas tareas, como PointNet++ [1], necesitan grandes conjuntos etiquetados, y etiquetar nubes de puntos a mano es lento y caro. Por eso crece el interés en entrenar con datos sintéticos: modelos entrenados con árboles simulados igualan o superan a los entrenados con datos reales de otros sitios [2, 3], y el preentrenamiento sintético reduce drásticamente la cantidad de datos reales necesaria [4].

Sin embargo, la pregunta de *cómo* generar los árboles sintéticos, y cómo saber si se parecen a los reales, está poco estudiada. La literatura de gráficos por ordenador evalúa árboles por su

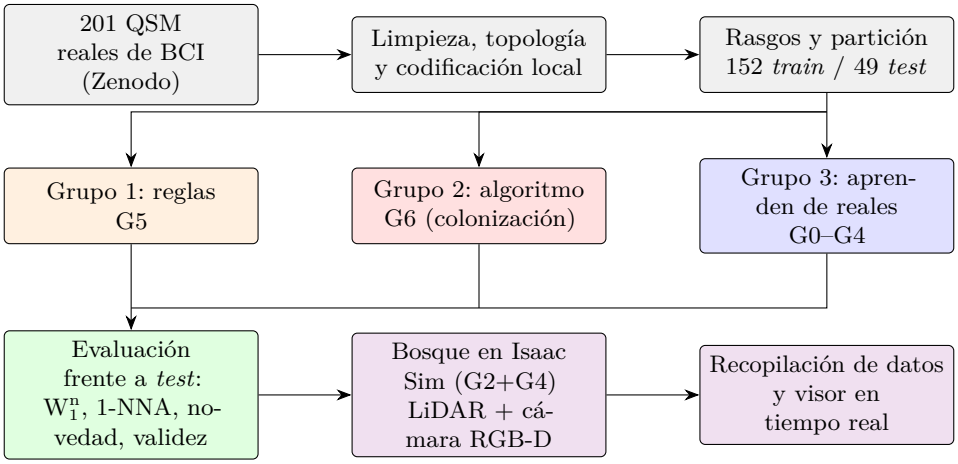


Figura 1: Flujo del trabajo: de los QSM reales a los generadores, su evaluación y el bosque virtual con sensores.

apariencia [5, 6]; la de teledetección evalúa los QSM por su error de volumen frente a medidas destructivas. Pocos trabajos comparan la *estructura* (topología, ángulos, jerarquía de ramas) de árboles sintéticos con la de árboles reales usando métricas comunes.

Este trabajo aborda ese hueco con un conjunto público de 201 QSM de árboles tropicales [7]. Sus contribuciones son:

1. Un *pipeline* reproducible de lectura, limpieza y codificación local (tipo tortuga) de QSM que permite modificar, perturbar e injertar ramas reconstruyendo las uniones automáticamente.

2. Ocho generadores que cubren las tres familias de la literatura: reglas, algoritmos de crecimiento y métodos que aprenden de árboles reales.

3. Un protocolo de evaluación con separación estricta entrenamiento/prueba por árbol, que mide fidelidad, novedad y validez botánica, siempre con la referencia de la variabilidad real–real.

4. Un bosque virtual en NVIDIA Isaac Sim [8] construido con los mejores generadores, con ramas continuas a partir de los QSM, en dos versiones (solo árboles y bosque ambientado). Incluye LiDAR y cámara de profundidad simulados, recopilación de datos etiquetados en ambas versiones y visualización en tiempo real.

La figura 1 resume el flujo completo.

2. Trabajo relacionado

Organizamos la literatura en tres familias de métodos, más la evaluación de datos sintéticos y su uso en percepción.

Reglas que cumplen los árboles reales. Hallé, Oldeman y Tomlinson describieron 23 *modelos arquitecturales* que resumen cómo crecen los árboles, a partir precisamente de árboles tropicales [9]. La regla de Leonardo y el modelo de tubería postulan que el área de una rama madre se reparte entre sus hijas [10]. La teoría metabólica (WBE) predice ramificaciones simétricas con exponentes fijos [11, 12], pero en árboles tropicales escaneados las hijas son asimétricas y los exponentes se desvían [13, 14]; solo los árboles dominantes del dosel se acercan a WBE [15]. En el mismo sitio de nuestros datos existen alometrías de altura y copa por especie, ajustadas con casi diez mil árboles [16].

Algoritmos que generan árboles. Los L-systems describen el crecimiento como reescritura de cadenas [17, 18]. AMAPsim [19] y GreenLab [20] incorporan conocimiento botánico y reparto de recursos. En gráficos por ordenador, *space colonization* [21] hace crecer las ramas hacia puntos libres de una envolvente, y los modelos autoorganizados [22] añaden competencia por luz entre yemas.

Métodos que aprenden de árboles reales. El modelado procedural inverso busca los parámetros de un generador que reproducen árboles de entrada; Šťava et al. [23] lo hacen con MCMC sobre árboles reconstruidos, y Guo et al. [24] infieren directamente las reglas de un L-system. Como complemento, Åkerblom et al. [25] insertan hojas sin intersecciones en QSM, y Treegraph [14] extrae arquitectura de ramas a partir de nubes TLS.

Evaluación de datos sintéticos. La literatura de modelos generativos distingue fidelidad, diversidad y utilidad. Para fidelidad se usan tests de dos muestras basados en clasificadores [26], en particular la precisión 1-NN *leave-one-out* (1-NNA), popularizada para nubes de puntos por PointFlow [27], y las métricas MMD/COV [28]. Para diversidad, la precisión y exhaustividad de variedades [29]. La utilidad se mide entrenando con sintéticos y probando con reales (TSTR) [30]; van Breugel et al. [31] advierten de que evaluar con datos del mismo generador sobreestima el rendimiento. Usamos además distancias de Wasserstein entre distribuciones de rasgos [32]. Para árboles, ICTree [5] propone métricas perceptuales, y DeepTree [6] compara longitudes y número de ramas.

Datos sintéticos para percepción forestal. Bryson et al. entrenan PointNet++ con árboles sintéticos y obtienen un rendimiento comparable o superior al de datos reales de otros sitios [2], y añaden adaptación de dominio para segmentar partes del árbol [3]. She et al. [4] muestran que la simulación física del LiDAR (HELIOS++ [33]) y la diversidad de escenas son los factores que más pesan (+37 y +39 puntos de F1), mientras que la cantidad de datos aporta menos (+5,5). En plantas industriales, simular el escaneo mejora el mIoU hasta un 7 % [34]. La aleatorización de dominio [35] es la estrategia general para cerrar la brecha simulación–realidad.

Hueco. No encontramos trabajos que ajusten generadores *por especie* a estadísticas de QSM tropicales ni que comparen las tres familias con métricas estructurales comunes. Tampoco hay estudios que relacionen el parecido estructural de los sintéticos con su utilidad para entrenar redes.

3. Datos

3.1. QSM de Barro Colorado Island

Usamos el conjunto de Krishna Moorthy et al. [7] (CC BY 4.0): 201 árboles de 53 especies escaneados con láser terrestre en 2019 en Barro Colorado Island (Panamá) y reconstruidos con TreeQSM 2.4.0 [36]. El estudio original evaluaba el efecto de las lianas, así que la selección de árboles no es aleatoria; anotamos esta condición como posible fuente de confusión. Cada QSM es una tabla de cilindros con inicio, eje, longitud, radio, padre y extensión (cuadro 1). El registro incluye además nubes de puntos separadas de madera y hoja.

3.2. Limpieza y topología

Para cada árbol conservamos solo la parte conectada al cilindro raíz (en promedio se descarta el 0,56 % de los cilindros, que son fragmentos sueltos). La rama, el orden y la posición de cada cilindro se recalculan desde `parent` y `extension`; el resultado coincide al 100 % con TreeQSM en los 201 árboles, y el mismo código se aplica a reales y sintéticos.

Tabla 1: Resumen de los 201 QSM reales (mediana [rango intercuartílico] salvo indicación).

Árboles / especies	201 / 53
Altura (m), media ± d.e.	28,5 ± 5,1
Cilindros por árbol	1785 [1115–3368]
Ramas por árbol	366 [217–646]
Orden máximo de rama	7 (3–10)
Radio de cilindro (cm)	2,7 [1,6–3,7]

Detectamos un artefacto de los datos reales: el 4,7 % de las uniones entre cilindros consecutivos de una misma rama tienen huecos de más de 20 cm, en 199 de los 201 árboles. Son errores de reconstrucción de TreeQSM, no del procesamiento, y los generadores que copian geometría real los heredan.

3.3. Partición entrenamiento/prueba

Separamos por árbol, estratificando por especie: 152 árboles de entrenamiento (*train*) y 49 de prueba (*test*), con las especies de menos de tres árboles asignadas a *train*. Todos los estadísticos de los generadores se ajustan solo con *train*, y los sintéticos se generan solo a partir de árboles de *train*. Los árboles de *test* se usan únicamente para evaluar.

4. Métodos

4.1. Codificación local de un QSM

Para modificar un árbol sin romper sus uniones, expresamos cada cilindro i en el marco ortonormal F_p de su padre p, como una tortuga de L-system:

$$\mathbf{d}_i = F_p^{\top}(\mathbf{s}_i - \mathbf{s}_p), \qquad (t_i, u_i, v_i) = \left(\frac{d_{i,z}}{\ell_p}, \frac{d_{i,x}}{r_p}, \frac{d_{i,y}}{r_p}\right), \qquad \mathbf{a}_i^{\mathrm{loc}} = F_p^{\top}\mathbf{a}_i, \tag{1}$$

donde $\mathbf{s}$ es el inicio, $\mathbf{a}$ el eje unitario, ℓ la longitud y r el radio. El marco del hijo se obtiene aplicando a F_p la rotación mínima que lleva $\mathbf{a}_p$ a $\mathbf{a}_i$ (forma de Rodrigues con el término $1/(1+c)$, numéricamente estable) y re-ortonormalizando en cada paso. La decodificación recorre el árbol en orden topológico. El error de ida y vuelta es de $\sim 10^{-7}$ m. Así, cambiar el radio, la longitud o el ángulo de una rama, o injertar un subárbol, desplaza automáticamente todo lo que cuelga de ella.

4.2. Rasgos

Con el mismo código para reales y sintéticos calculamos 22 rasgos de árbol (altura, DBH, volumen total, fracción de volumen en ramas, longitud total, número de ramas, orden máximo, ramas de orden 1 y 2, área y diámetro de copa por envolvente convexa, base de copa relativa, ángulo y cociente de diámetros medianos de las ramas de orden 1, densidad de hijas, regla de tubería, fracciones de volumen por orden y perfil de adelgazamiento del tronco a 30, 50, 70 y 90 % de su altura). También calculamos cinco rasgos por rama para los órdenes 1 y 2: ángulo de inserción, cociente de diámetros, cociente de longitudes, cenit y densidad de hijas.

En *train* medimos las reglas que usan los generadores:

- alometría de árbol $H \propto \mathrm{DBH}^{0,427}$;
- alometría de rama $\ell \propto r^{0,345}$ (residuo d.e. 0,79 en escala log);
- variabilidad natural del cociente de diámetros (d.e. log de 0,73 en orden 1 a 1,0 en orden 7);

Tabla 2: Generadores implementados. Del árbol fuente, cada generador toma lo que indica la última columna.

Grupo	Generador	Idea	Toma del fuente
3	G0 `iid_radius`	Radio de cada cilindro $\times\, U(0{,}9; 1{,}1)$, línea base del repositorio `qsm-variation`.	Todo
3	G1 `allometric`	Por rama: factores log-normales de media 1 en radio y longitud (ligada por la alometría), y ruido en ángulo de inserción y azimut; magnitud $\alpha\times$ variabilidad natural; escala global $H \propto \mathrm{DBH}^b$.	Todo
3	G2 `recombine`	Cada rama de orden 1 se sustituye con $p = 0{,}5$ por la de otro árbol de *train* de la misma especie, con altura relativa ($\pm 0{,}15$) y grosor ($\times 0{,}5$–2) parecidos; radio $\times k$, longitud $\times k^b$.	Tronco, resto de ramas
3	G3 `hybrid`	G2 seguido de G1 con $\alpha = 0{,}25$.	Tronco, resto de ramas
3	G4 `procedural`	Árbol nuevo rama a rama: cada rama sortea la «regla» completa de una rama real de la misma especie y orden (posición, cociente de diámetros, ángulos, curvatura, perfil de radio, residuo de longitud, n.º de hijas), entre ramas cuyo padre real tenía grosor parecido.	Especie, tamaño y perfil del tronco, n.º de ramas de orden 1
1	G5 `rules`	Árbol nuevo solo con medias y dispersiones por especie y orden: alometrías, perfil de adelgazamiento medio, filotaxis de 137,5°, ángulos, cenit, densidad de ramas y cociente de longitudes hija/madre $\leq$ percentil 95.	Especie, DBH
2	G6 `colonization`	*Space colonization* [21] en un elipsoide de copa; radios por la regla de Leonardo con $\Delta = 2{,}38$ ajustada al DBH.	Especie, envolvente de copa, longitud total

- variabilidad del ángulo de inserción (d.e. de 40° en orden 1 a 32° en orden 8);
- exponente de Leonardo Δ tal que $r_p^{\Delta} = \sum_c r_c^{\Delta}$: mediana $\Delta = 2{,}38$ sobre 25 728 bifurcaciones con $r_p > 2\,\mathrm{cm}$, cerca del valor clásico 2.

4.3. Generadores

Cada generador produce cinco árboles por árbol de *train* (760 en total). Los resumimos en la cuadro 2.

G1, perturbación alométrica. Para cada rama b de orden o, con $\sigma_r = \alpha\,\sigma_{\mathrm{ratio},o}$ y $\sigma_\ell = \alpha\,\sigma_{\mathrm{res},\ell}$:

$$r' = r\,e^{\varepsilon_r - \sigma_r^2/2}, \quad \ell' = \ell\,e^{\,b\,(\varepsilon_r + \sigma_r^2/2) + \varepsilon_\ell - (b^2\sigma_r^2 + \sigma_\ell^2)/2}, \quad \theta' = \theta + \alpha\,\sigma_{\theta,o}\,\xi, \tag{2}$$

con $\varepsilon_r \sim \mathcal{N}(0, \sigma_r^2)$, $\varepsilon_\ell \sim \mathcal{N}(0, \sigma_\ell^2)$ y $\xi \sim \mathcal{N}(0,1)$. Los términos de corrección mantienen la media; sin ellos, el ruido log-normal infla el tamaño del árbol. Además se impone que una rama no supere el radio de su padre, salvo que ya lo hiciera en el original.

G2, recombinación. Si el grosor de la rama sustituida es r_t y el de la donante r_d, la donante se reescala con $k = r_t/r_d$: radios $\times k$ y longitudes $\times k^b$, con $b = 0{,}345$. Conserva su ángulo de inserción y adopta el azimut de la rama sustituida. En especies con un solo árbol en *train* se usan donantes de otras especies, lo que afecta al 16 % de los sintéticos.

G4, procedural aprendido. Es una versión sin ajuste inverso del enfoque de Šťava et al. [23]. En lugar de muestrear cada parámetro por separado, lo que rompe sus correlaciones, se sortea la fila completa de una rama real. Para evitar aplicar la regla de una rama fina sobre un padre grueso, se toman 32 candidatas al azar y se elige entre las 3 cuyo padre real tenía el grosor más parecido. La curvatura se impone interpolando (*slerp*) entre la dirección inicial y la final de la rama real, más una ondulación con la misma amplitud media medida en las uniones reales.

G5, reglas. Al principio aplicamos Leonardo en cada lateral para adelgazar los ejes, pero el tronco quedaba casi sin grosor (radio a media altura del 9 % del de 1,3 m). En los QSM, la suma de áreas de las hijas *supera* la de la madre (mediana del cociente 1,34), así que G5 usa el perfil de adelgazamiento medio de la especie, y reservamos Leonardo para G6.

G6, space colonization. Los puntos de atracción se distribuyen uniformemente en un elipsoide con la altura, la base y el diámetro de copa del fuente, en número proporcional a su longitud total (un punto por metro). El radio de influencia es tres veces la separación media entre puntos, la distancia de eliminación es 0,5 m, el segmento mide 0,25 m y hay un tropismo vertical de 0,15. Los radios siguen $r = r_{\text{tip}}\, w^{1/\Delta}$, donde w es el número de puntas aguas abajo, ajustado al DBH.

Los únicos parámetros elegidos a mano fueron la ondulación de G4 y la densidad de puntos de G6. Comprobamos que la elección es la misma usando medianas de *train* o de *test*, de modo que no hay fuga de información hacia la evaluación.

4.4. Métricas

Sean $\mathcal{S}$ los sintéticos y $\mathcal{T}$ los árboles de *test*. Todas las métricas se reportan junto a la referencia real–real, que compara *train* con *test*: un sintético es «tan real como lo real» si su distancia a *test* se parece a la de *train* a *test*.

Wasserstein normalizada. Para cada rasgo j (en escala logarítmica los de tamaño):

$$\mathrm{W}_1^{\mathrm{n}}(j) = \frac{W_1\left(F_j^{\mathcal{S}}, F_j^{\mathcal{T}}\right)}{\sigma_j^{\text{real}}}. \tag{3}$$

1-NNA. Con los vectores de rasgos estandarizados, se juntan $|\mathcal{T}| = 49$ árboles de test y 49 sintéticos al azar, y se mide la precisión *leave-one-out* de un clasificador 1-NN que predice si un árbol es real o sintético [26, 27]. Un 50 % significa indistinguible. Promediamos 50 submuestreos y, para las mezclas, 5 semillas.

Novedad. Mide si un sintético es un árbol distinto o una copia de su fuente:

$$\text{novedad} = \frac{\text{med}_{s\in\mathcal{S}} \|\mathbf{z}_s - \mathbf{z}_{\text{fuente}(s)}\|}{\text{med}_{i\in\text{train}}\, \text{mín}_{j\neq i} \|\mathbf{z}_i - \mathbf{z}_j\|}. \tag{4}$$

Vale 0 para una copia y 1 si el sintético se aleja de su fuente lo mismo que un árbol real de su vecino real más cercano.

Validez botánica. Porcentaje de ramas más gruesas que su madre y mediana de la regla de tubería $\sum r_c^2 / r_p^2$.

Tabla 3: Evaluación frente a los 49 árboles de *test* (760 sintéticos por generador). En negrita, el mejor valor entre los sintéticos.

	Real (ref.)	G0	G1 α=0,25	G1 α=0,5	G2	G3	G4	G5	G6
Grupo	–	3	3	3	3	3	3	1	2
1-NNA (%) ↓	45,5	**54,2**	57,0	65,1	55,9	55,9	63,2	89,2	97,8
Novedad	1,00	0,31	0,38	0,72	0,73	0,81	1,60	1,98	2,13
W_1^n árbol (media) ↓	0,202	**0,201**	0,226	0,310	0,209	0,233	0,300	0,632	0,820
W_1^n rama (media) ↓	0,039	**0,042**	0,052	0,075	0,049	0,061	0,063	0,096	0,312
Ramas > madre (%)	5,8	12,0	5,1	5,2	5,7	5,0	5,5	0,0	0,0
Tubería (mediana)	1,34	1,35	1,33	1,30	1,34	1,33	1,38	1,21	1,10

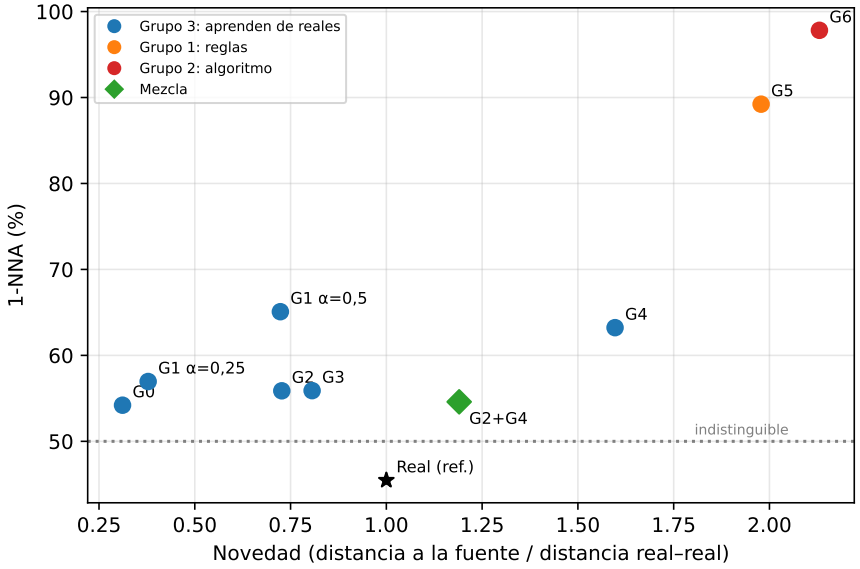


Figura 2: Fidelidad (1-NNA, más bajo es mejor; 50 % = indistinguible) frente a novedad. La estrella es la referencia real–real. Cuanto menos se apoya un método en ramas reales, más novedoso es y menos se parece a lo real.

5. Resultados

5.1. Comparación de generadores

La cuadro 3 y la figura 2 muestran los resultados, y la figura 3 un ejemplo visual.
Lo más relevante:

- **G0** tiene la menor distancia porque es casi una copia (novedad 0,31), y además *duplica* el porcentaje de ramas más gruesas que su madre (12 % frente a 5,8 %). Rompe una regla botánica básica sin crear árboles nuevos.
- **G1** muestra el compromiso fidelidad–novedad. Con $\alpha = 0{,}5$ la copa se ensancha un 22 %, porque el ruido independiente por rama no conoce la forma global de la copa.
- **G2** es el mejor equilibrio: novedad 0,73 con W_1^n de árbol (0,209) casi igual a la referencia real (0,202).
- **G4** es el único generador realista que crea árboles realmente nuevos (novedad 1,60). Sus errores principales son el reparto de volumen entre órdenes y el número de ramas de orden 2 (figura 4).

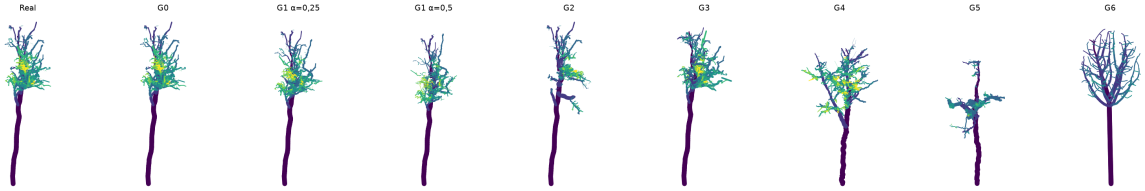


Figura 3: Árbol real (*Trichilia tuberculata*, TRITUB_213134) y un sintético de cada generador, en vista lateral. El color indica el orden de rama.

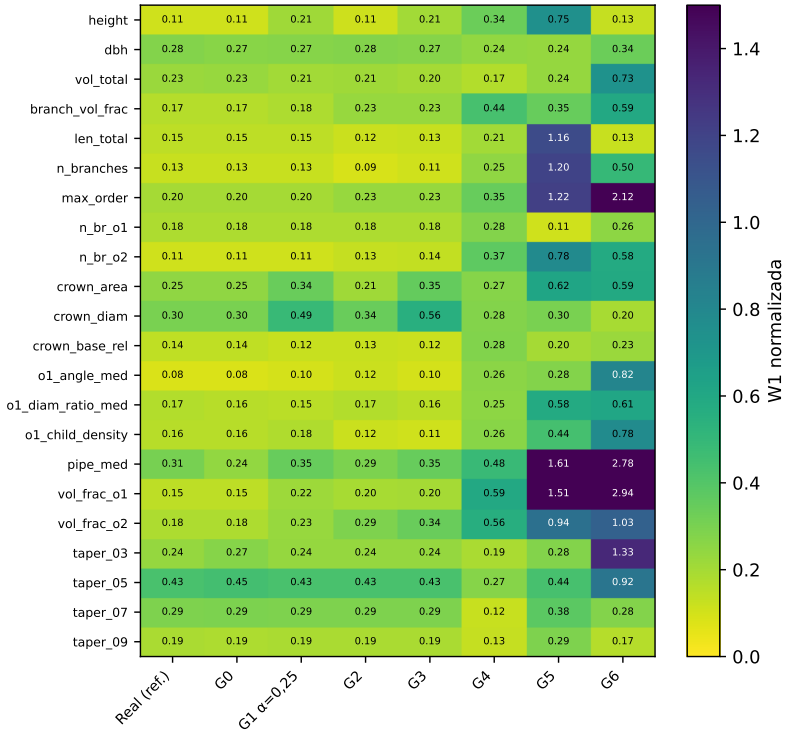


Figura 4: Distancia de Wasserstein normalizada por rasgo de árbol frente a *test*. La primera columna es la referencia real–real.

- **G5** acierta el tamaño (altura, DBH, copa), pero tiene un tercio de las ramas (128 frente a 365) y menos órdenes (5 frente a 7). Las reglas medias, aplicadas por separado, no reproducen la ramificación densa y desordenada de los QSM.
- **G6** acierta la longitud total y la copa, que fija la envolvente, pero produce una arquitectura de gráficos por ordenador: jerarquía poco profunda (orden máximo 4), tronco sin adelgazar y ramas en abanico.

5.2. Combinaciones de generadores

Mezclamos generadores a partes iguales, manteniendo 760 árboles en total (cuadro 4). La mezcla G2+G4 tiene la misma fidelidad que G2 solo ($54{,}6 \pm 1{,}0\,\%$ frente a $54{,}5 \pm 0{,}5\,\%$; la diferencia está dentro del ruido entre semillas), pero su novedad es mucho mayor (1,19 frente a 0,73). Añadir G3 no aporta nada, y mezclar con G5 o G6 degrada la fidelidad.

Tabla 4: Mezclas de generadores (760 árboles, partes iguales). Media de 5 semillas (submuestreos distintos); para 1-NNA, media ± d.e.

Conjunto	1-NNA (%)	Novedad	W_1^n árbol
Real *train* (ref.)	$47{,}5 \pm 0{,}4$	1,00	0,20
G2	$54{,}5 \pm 0{,}5$	0,73	0,21
G4	$60{,}7 \pm 1{,}4$	1,60	0,30
G2 + G4	$\mathbf{54{,}6 \pm 1{,}1}$	**1,19**	0,23
G2 + G3 + G4	$54{,}8 \pm 1{,}1$	1,01	0,23
G2 + G5	$64{,}1 \pm 0{,}6$	1,41	0,36
G2 + G6	$67{,}7 \pm 0{,}9$	1,66	0,47

6. Bosque virtual con sensores en Isaac Sim

Con la mejor combinación (G2+G4) construimos un bosque virtual para generar datos de sensores, que es el paso necesario para medir la utilidad de los sintéticos (TSTR). Construimos dos versiones del mismo bosque, con los mismos árboles, posiciones y recorrido:

- **v1, solo árboles:** los 24 árboles sobre suelo plano y cielo de campo;
- **v2, bosque ambientado:** los mismos árboles, con terreno, sotobosque, madera muerta, rocas y un anillo exterior de árboles.

Así las diferencias entre los datos de ambas se deben solo a la ambientación.

6.1. Árboles: de cilindros a ramas continuas

Elegimos 12 árboles G2 y 12 G4 con fuentes distintas y los colocamos en una parcela de 60×60 m mediante muestreo de disco de Poisson (separación mínima de 7 m) con rotación aleatoria.

Una primera versión convertía cada cilindro QSM en un tubo independiente. El resultado era visualmente pobre: se veían los cilindros, y los huecos de TreeQSM (apartado 3.2) dejaban ramas desconectadas. La versión final construye **un tubo continuo por rama**:

1. El eje de la rama une el inicio del primer cilindro, el punto medio de cada unión (lo que cierra los huecos) y el final del último.
2. Cada rama hija empieza en el punto del eje de su madre más cercano a su base, así que nace desde dentro de ella.
3. El eje se alisa con un filtro laplaciano [37], con más iteraciones cuanto más gruesa es la rama (entre 1 y 12, $\propto r/2$ cm), lo que elimina el zigzag de reconstrucción de los troncos sin cambiar la forma de las ramas finas. Después se suaviza con dos iteraciones de Chaikin [38].
4. Los anillos de vértices (de 6 a 20 lados según el radio) se orientan con marcos de transporte paralelo [39], que evitan la torsión, y se comparten entre segmentos consecutivos.
5. Cada rama tiene normales suaves y coordenadas UV a lo largo de su longitud, de modo que la corteza sigue su dirección. La punta se cierra en cono.

Cada árbol tiene una malla de tronco (orden 0) y otra de ramas, etiquetadas semánticamente (`trunk`, `branch`). El bosque v1 suma 2,36 M caras en formato OpenUSD [40]. Los materiales son PBR con texturas escaneadas de corteza [41].



Figura 5: El mismo bosque en Isaac Sim: v1, solo árboles (izquierda), y v2, bosque ambientado (derecha). Arriba, vista desde el recorrido de la plataforma; en medio, a la altura de una persona; abajo, vista aérea.

6.2. Ambientación de bosque (v2)

Para dar sensación de bosque añadimos los siguientes elementos, todos con recursos CC0 de Poly Haven [41]:

- **Terreno:** suma de 14 ondas de 12 a 90 m de longitud (desnivel de 2,4 m en la parcela; d.e. 0,49 m), con textura de hojarasca. Los árboles y la plataforma siguen su altura.
- **Horizonte de bosque:** un anillo de 110 árboles sintéticos G2/G4 adicionales, de fuentes no usadas en la parcela, entre 34 y 95 m del centro, en una versión ligera de la malla (mitad de lados y sin ramitas de menos de 6 mm). Lo completa un cielo HDRI de sendero de selva.
- **Sotobosque** en un radio de 60 m: 4 431 plantas (helechos, *Calathea*, *Anthurium* y arbustos).
- **Madera muerta:** 835 objetos (tocones, troncos caídos, raíces, corteza y ramas secas).
- **Rocas:** 23 rocas con musgo.

Las plantas y los objetos se instancian (*PointInstancer*) y se etiquetan como `understory`, `deadwood` y `rock`. Ninguno se coloca sobre el recorrido de la plataforma. La parte no instanciada de v2 suma 5,40 M caras. La figura 5 compara ambas versiones.

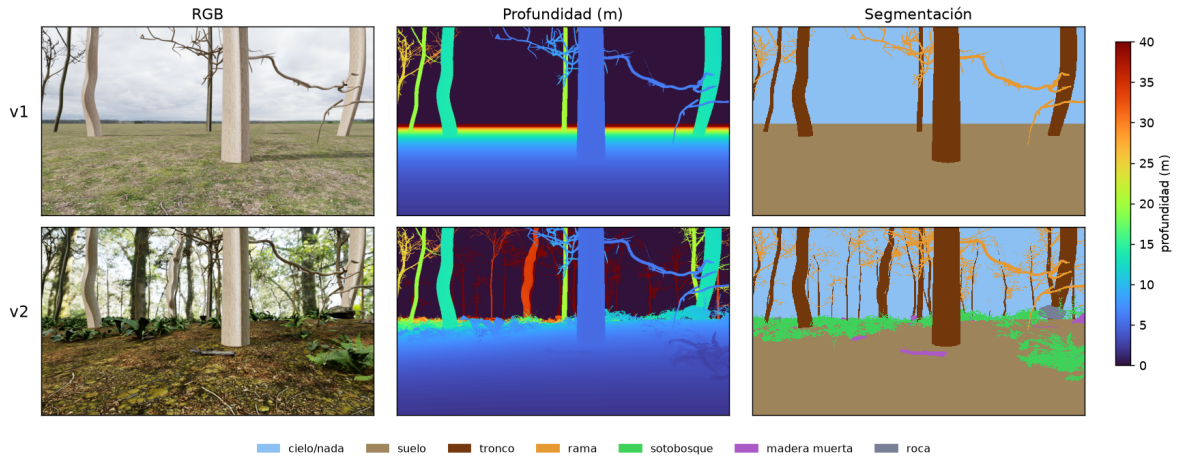


Figura 6: Cámara RGB-D en el mismo instante y la misma pose en v1 (arriba) y v2 (abajo): RGB, profundidad y segmentación semántica.

6.3. Sensores

Una plataforma recorre a 1 m/s un circuito cerrado de 83 m, de la forma $r(\theta) = r_0 + A\sin(k\theta + \varphi)$. Sus parámetros se eligen para pasar cerca de los árboles sin acercarse a menos de 2,5 m de ningún tronco. La plataforma lleva dos sensores:

- **LiDAR rotatorio** Ouster OS1 (REV6, 128 canales, 1024 columnas, 10 Hz) [42] a 1,8 m del suelo, simulado con el trazado de rayos RTX de Isaac Sim 6.2 [8]. Cada barrido se decodifica de coordenadas esféricas (acimut, elevación, rango) a cartesianas y se transforma a coordenadas de mundo. Mediante el identificador de objeto de cada retorno, cada punto recibe su clase y su árbol.
- **Cámara RGB-D** a 1,2 m con la óptica de una Intel RealSense D455 (87° de campo horizontal, 848 × 480) [43]: RGB, profundidad al plano imagen y segmentación semántica.

6.4. Recopilación y visualización

A 10 Hz, cada fotograma (último barrido LiDAR, imágenes, poses de plataforma, LiDAR y cámara, e intrínsecos) se guarda en un archivo comprimido con escritura atómica. Un proceso independiente, basado en Rerun [44], lee los fotogramas según aparecen y los muestra en tiempo real. La vista 3D incluye:

- el barrido actual coloreado por clase;
- el mapa acumulado submuestreado por vóxeles de 15 cm;
- el esqueleto «verdad» de cada árbol;
- la trayectoria y la cámara con su profundidad proyectada.

También hay vistas 2D de RGB, profundidad y segmentación, y series temporales de puntos por clase. El mismo visor reproduce grabaciones, lo que sirve como respaldo para una demostración.

6.5. Resultados: datos de v1 y v2

Grabamos una vuelta completa (84 s simulados, 840 fotogramas) en cada versión, en una RTX 5070 Ti (cuadro 5, figuras 6 y 7).

Tabla 5: Datos de sensores en una vuelta completa por las dos versiones del bosque (mismos árboles y mismo recorrido).

	v1: solo árboles	v2: bosque ambientado
Tiempo real por vuelta (84 s simulados)	144 s (0,58×)	196 s (0,43×)
Tamaño de la grabación	1,7 GB	2,6 GB
Puntos LiDAR por barrido	60 098 ± 1 166	77 823 ± 917
LiDAR, % de puntos		
suelo	83,0	52,0
tronco	15,8	17,7
rama	1,3	7,2
sotobosque	–	21,7
madera muerta	–	1,3
roca	–	0,2
Cámara, % de píxeles		
cielo / fondo	45,1	36,2
suelo	47,5	40,1
tronco	6,6	9,2
rama	0,8	4,8
sotobosque	–	9,0
madera muerta + roca	–	0,7
Árboles de la parcela vistos por barrido (de 24)	23,9	23,8
Puntos de tronco por árbol de la parcela visto	398	382 (−4 %)
Error del suelo LiDAR frente al terreno (mediana)	1 mm	4 mm
Distancia mínima del LiDAR a un objeto	1,76 m	1,73 m

Coherencia geométrica. En ambas versiones el suelo medido por el LiDAR coincide con el terreno (error mediano de 1–4 mm). Ningún objeto queda a menos de 1,7 m del sensor, así que el recorrido está despejado. En v1, los puntos de tronco de cada árbol caen a una mediana de 0,25–1 m de la posición de su base (hasta 2,7 m en troncos inclinados).

Qué cambia con la ambientación.

- **Escena más compleja:** el sotobosque pasa a ser la segunda clase del LiDAR (21,7 %) y el suelo baja del 83 al 52 %. Una red entrenada en v1 nunca habría visto vegetación baja, que en un bosque real ocupa una parte importante del escaneo cerca del suelo.
- **Más copa visible:** la proporción de ramas sube de 1,3 a 7,2 %, sobre todo por las copas del anillo exterior, que el LiDAR ve a distancia. Desde dentro de la parcela, el campo vertical de ±21° apenas alcanza las copas a 15–30 m de altura.
- **Poca oclusión de troncos:** los 24 árboles de la parcela siguen viéndose en cada barrido, y los puntos de tronco por árbol solo bajan un 4 %, porque el sotobosque es bajo.
- **Coste:** v2 tarda un 36 % más en simularse y produce un 53 % más de datos.

7. Discusión

Escalera fidelidad–novedad. Los resultados confirman una escalera clara: cuanto más se apoya un generador en geometría real, más fiel y menos novedoso es (G0 → G2 → G4), y los métodos sin datos reales (G5, G6) son fácilmente distinguibles. Esto coincide con lo que la literatura sugiere sobre la importancia de la diversidad [4]. También indica que los generadores de gráficos por ordenador sin calibrar no deberían usarse como sustitutos de árboles tropicales reales. Siguen siendo útiles, sin embargo, como *controles* de baja similitud para estudiar si el parecido predice la utilidad.

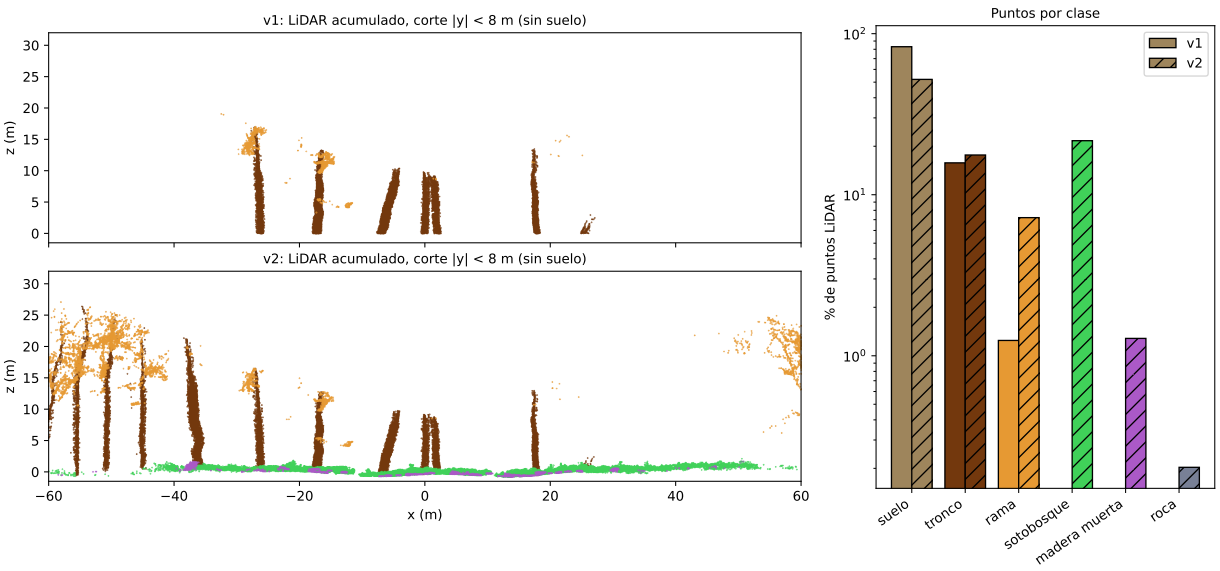


Figura 7: LiDAR Ouster OS1 simulado: corte lateral del mapa acumulado durante una vuelta (sin suelo) en v1 y v2, y porcentaje de puntos por clase (escala logarítmica).

Por qué G4 funciona mejor que G5. Ambos construyen árboles desde cero con información de la especie. La diferencia es que G4 conserva la distribución *conjunta* de los rasgos de cada rama y condiciona por el grosor del padre, mientras que G5 muestrea rasgos independientes alrededor de su media. Esto ilustra la advertencia de que muestrear distribuciones marginales produce árboles raros aunque cada rasgo individual cuadre.

Limitaciones.

- Los rasgos se calculan sobre QSM, que son a su vez reconstrucciones con sesgos (por ejemplo, la regla de tubería de 1,34 y los huecos entre cilindros), no sobre los árboles reales.
- Con 49 árboles de *test*, el 1-NNA tiene poca potencia estadística, así que «indistinguible» puede significar solo «pocos datos».
- No se comprueban intersecciones entre ramas, y los árboles no tienen hojas.
- La alometría publicada de BCI [16] no se pudo descargar porque el repositorio exige autenticación. Las reglas se ajustaron con nuestros datos de *train*.
- En Isaac, la profundidad de la cámara es ideal (sin el ruido estereoscópico de una D455) y el LiDAR no modela la reflectancia del material de forma calibrada.
- En v2, el bosque que se ve al fondo de la cámara es la fotografía del cielo HDRI: no tiene geometría, así que aparece sin profundidad y etiquetado como fondo, y el LiDAR no lo ve. Para entrenar con la cámara conviene un cielo sin árboles o enmascarar el fondo.
- El sotobosque y la madera muerta son modelos genéricos (CC0), no especies medidas en BCI. La textura del suelo se repite visiblemente desde el aire.
- Todavía no hemos medido la utilidad de los sintéticos para entrenar redes.

8. Conclusiones y trabajo futuro

Implementamos y evaluamos ocho generadores de árboles tropicales sintéticos de las tres familias de la literatura con un protocolo que separa estrictamente *train* y *test*. La recombinación de ramas

de la misma especie (G2) es la más fiel. El generador procedural que sortea reglas de ramas reales (G4) es el único que crea árboles realmente nuevos con fidelidad razonable. Su mezcla G2+G4 ofrece el mejor equilibrio. Con ella construimos un bosque virtual en Isaac Sim, en una versión solo con árboles y otra ambientada, con LiDAR y cámara RGB-D simulados que generan datos etiquetados por clase y se visualizan en tiempo real. La ambientación cambia sustancialmente la distribución de clases que ven los sensores.

Los siguientes pasos son:

1. Entrenar redes de segmentación (suelo/tronco/rama) con datos reales, sintéticos (v1 y v2) y mixtos, y evaluarlas sobre las nubes reales de BCI (TSTR). Comparar v1 y v2 indicará cuánto aporta la ambientación.

2. Comprobar si el 1-NNA de cada generador predice su utilidad.

3. Añadir hojas [25], aprendidas de las nubes reales de hoja de BCI, y compararlas con ellas (trabajo iniciado, fuera del alcance de este documento).

4. Simular el escaneo terrestre real con HELIOS++ [33] o con el LiDAR RTX colocado como en la campaña original.

5. Ajustar G4 por modelado inverso [23] para corregir el reparto de volumen entre órdenes.

Reproducibilidad. El código, los scripts que regeneran todas las tablas y figuras de este documento y las instrucciones están en `https://github.com/Nosl3n/Synthetic_Tree_G`: `synth_trees/` (generadores y evaluación) e `isaac_forest/` (geometría, escenas v1 y v2, sensores y visor). Los datos de entrada son públicos [7]. Todas las semillas están fijadas: una ejecución desde cero en un entorno limpio reproduce bit a bit los 6 080 árboles sintéticos y todas las métricas de las cuadros 3 y 4. Las grabaciones de sensores, los videos y las escenas se distribuyen aparte por su tamaño.

Referencias

[1] Charles R. Qi, Li Yi, Hao Su y Leonidas J. Guibas. «PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space». En: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017. arXiv: 1706.02413.

[2] Mitch Bryson, Feiyu Wang y James Allworth. «Using Synthetic Tree Data in Deep Learning-Based Tree Segmentation Using LiDAR Point Clouds». En: *Remote Sensing* 15.9 (2023), pág. 2380. DOI: 10.3390/rs15092380.

[3] Mitch Bryson et al. «Domain adaptation of deep neural networks for tree part segmentation using synthetic forest trees». En: *ISPRS Open Journal of Photogrammetry and Remote Sensing* 14 (2024). URL: https://doaj.org/article/953802df7d1b400a8be548c51f1f3a41.

[4] Yihang She, Andrew Blake, David Coomes y Srinivasan Keshav. «Scaling Up Forest Vision with Synthetic Data». En: *International Journal of Computer Vision* (2026). DOI: 10.1007/s11263-026-02923-y. arXiv: 2509.11201.

[5] Tomas Polasek, David Hrusa, Bedrich Benes y Martin Čadík. «ICTree: Automatic Perceptual Metrics for Tree Models». En: *ACM Transactions on Graphics* 40.6 (2021), 230:1-230:15. DOI: 10.1145/3478513.3480519.

[6] Xiaochen Zhou et al. «DeepTree: Modeling Trees with Situated Latents». En: *IEEE Transactions on Visualization and Computer Graphics* (2023). arXiv: 2305.05153.

[7] Sruthi M. Krishna Moorthy et al. *Datasets to quantify the impact of lianas on 3D tree structure and biomass*. Licencia CC BY 4.0. Zenodo, 2022. DOI: 10.5281/zenodo.6981485.

[8] NVIDIA. *Isaac Sim 6.2: robotics simulation and synthetic data generation*. https://docs.isaacsim.omniverse.nvidia.com. Consultado el 30/10/2026. 2026.

[9] Francis Hallé, Roelof A. A. Oldeman y P. Barry Tomlinson. *Tropical Trees and Forests: An Architectural Analysis*. Berlin: Springer, 1978. DOI: 10.1007/978-3-642-81190-6.

[10] Kichiro Shinozaki, Kyoji Yoda, Kazuo Hozumi y Tatuo Kira. «A quantitative analysis of plant form—the pipe model theory. I. Basic analyses». En: *Japanese Journal of Ecology* 14.3 (1964), págs. 97-105.

[11] Geoffrey B. West, James H. Brown y Brian J. Enquist. «A General Model for the Origin of Allometric Scaling Laws in Biology». En: *Science* 276.5309 (1997), págs. 122-126. DOI: 10.1126/science.276.5309.122.

[12] Geoffrey B. West, James H. Brown y Brian J. Enquist. «A general model for the structure and allometry of plant vascular systems». En: *Nature* 400 (1999), págs. 664-667. DOI: 10.1038/23251.

[13] Alvaro Lau et al. «Estimating architecture-based metabolic scaling exponents of tropical trees using terrestrial LiDAR and 3D modelling». En: *Forest Ecology and Management* 439 (2019), págs. 132-145. DOI: 10.1016/j.foreco.2019.02.019.

[14] Wanxin Yang, Phil Wilkes, Matheus B. Vicari, Kate Hand, Kim Calders y Mathias Disney. «Treegraph: tree architecture from terrestrial laser scanning point clouds». En: *Remote Sensing in Ecology and Conservation* (2024). DOI: 10.1002/rse2.399.

[15] Olivier Martin-Ducup et al. «Terrestrial laser scanning reveals convergence of tree architecture with increasingly dominant crown canopy position». En: *Functional Ecology* 34.12 (2020), págs. 2442-2452. DOI: 10.1111/1365-2435.13678.

[16] Isabel Martínez Cano, Helene C. Muller-Landau, S. Joseph Wright, Stephanie A. Bohlman et al. «Tropical tree height and crown allometries for the Barro Colorado Nature Monument, Panama: a comparison of alternative hierarchical models incorporating interspecific variation in relation to life history traits». En: *Biogeosciences* 16 (2019), págs. 847-862. DOI: 10.5194/bg-16-847-2019.

[17] Aristid Lindenmayer. «Mathematical models for cellular interactions in development I. Filaments with one-sided inputs». En: *Journal of Theoretical Biology* 18.3 (1968), págs. 280-299. DOI: 10.1016/0022-5193(68)90079-9.

[18] Przemyslaw Prusinkiewicz y Aristid Lindenmayer. *The Algorithmic Beauty of Plants*. New York: Springer, 1990. DOI: 10.1007/978-1-4613-8476-2.

[19] Jean-François Barczi et al. «AmapSim: A Structural Whole-plant Simulator Based on Botanical Knowledge and Designed to Host External Functional Models». En: *Annals of Botany* 101.8 (2008), págs. 1125-1138. DOI: 10.1093/aob/mcm194.

[20] Hong-Ping Yan, Meng Zhen Kang, Philippe de Reffye y Michael Dingkuhn. «A Dynamic, Architectural Plant Model Simulating Resource-dependent Growth». En: *Annals of Botany* 93.5 (2004), págs. 591-602. DOI: 10.1093/aob/mch078.

[21] Adam Runions, Brendan Lane y Przemyslaw Prusinkiewicz. «Modeling Trees with a Space Colonization Algorithm». En: *Eurographics Workshop on Natural Phenomena*. 2007, págs. 63-70. DOI: 10.2312/NPH/NPH07/063-070.

[22] Wojciech Palubicki et al. «Self-organizing tree models for image synthesis». En: *ACM Transactions on Graphics* 28.3 (2009), 58:1-58:10. DOI: 10.1145/1531326.1531364.

[23] Ondřej Šťava et al. «Inverse Procedural Modelling of Trees». En: *Computer Graphics Forum* 33.6 (2014), págs. 118-131. DOI: 10.1111/cgf.12282.

[24] Jianwei Guo et al. «Inverse Procedural Modeling of Branching Structures by Inferring L-Systems». En: *ACM Transactions on Graphics* 39.5 (2020), 155:1-155:13. DOI: 10.1145/3394105.

[25] Markku Åkerblom, Pasi Raumonen et al. «Non-intersecting leaf insertion algorithm for tree structure models». En: *Interface Focus* 8.2 (2018), pág. 20170045. DOI: 10.1098/rsfs.2017.0045.

[26] David Lopez-Paz y Maxime Oquab. «Revisiting Classifier Two-Sample Tests». En: *International Conference on Learning Representations (ICLR)*. 2017. arXiv: 1610.06545.

[27] Guandao Yang, Xun Huang, Zekun Hao, Ming-Yu Liu, Serge Belongie y Bharath Hariharan. «PointFlow: 3D Point Cloud Generation with Continuous Normalizing Flows». En: *IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, págs. 4541-4550. arXiv: 1906.12320.

[28] Panos Achlioptas, Olga Diamanti, Ioannis Mitliagkas y Leonidas Guibas. «Learning Representations and Generative Models for 3D Point Clouds». En: *International Conference on Machine Learning (ICML)*. 2018, págs. 40-49. arXiv: 1707.02392.

[29] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen y Timo Aila. «Improved Precision and Recall Metric for Assessing Generative Models». En: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019. arXiv: 1904.06991.

[30] Cristóbal Esteban, Stephanie L. Hyland y Gunnar Rätsch. *Real-valued (Medical) Time Series Generation with Recurrent Conditional GANs*. 2017. arXiv: 1706.02633.

[31] Boris van Breugel, Zhaozhi Qian y Mihaela van der Schaar. «Synthetic Data, Real Errors: How (Not) to Publish and Use Synthetic Data». En: *International Conference on Machine Learning (ICML)*. 2023. arXiv: 2305.09235.

[32] Aaditya Ramdas, Nicolás García Trillos y Marco Cuturi. «On Wasserstein Two-Sample Testing and Related Families of Nonparametric Tests». En: *Entropy* 19.2 (2017), pág. 47. DOI: 10.3390/e19020047.

[33] Lukas Winiwarter et al. «Virtual laser scanning with HELIOS++: A novel take on ray tracing-based simulation of topographic full-waveform 3D laser scanning». En: *Remote Sensing of Environment* 269 (2022), pág. 112772. DOI: 10.1016/j.rse.2021.112772.

[34] Florian Noichl et al. «Enhancing point cloud semantic segmentation in the data-scarce domain of industrial plants through synthetic data». En: *Computer-Aided Civil and Infrastructure Engineering* (2024). DOI: 10.1111/mice.13153.

[35] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba y Pieter Abbeel. «Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World». En: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, págs. 23-30. DOI: 10.1109/IROS.2017.8202133.

[36] Pasi Raumonen, Mikko Kaasalainen, Markku Åkerblom et al. «Fast Automatic Precision Tree Models from Terrestrial Laser Scanner Data». En: *Remote Sensing* 5.2 (2013), págs. 491-520. DOI: 10.3390/rs5020491.

[37] Gabriel Taubin. «A Signal Processing Approach to Fair Surface Design». En: *Proceedings of SIGGRAPH '95*. 1995, págs. 351-358. DOI: 10.1145/218380.218473.

[38] George M. Chaikin. «An algorithm for high-speed curve generation». En: *Computer Graphics and Image Processing* 3.4 (1974), págs. 346-349. DOI: 10.1016/0146-664X(74)90028-8.

[39] Richard L. Bishop. «There is More than One Way to Frame a Curve». En: *The American Mathematical Monthly* 82.3 (1975), págs. 246-251. DOI: 10.2307/2319846.

[40] Pixar Animation Studios. *Universal Scene Description (OpenUSD)*. https://openusd.org. Versión 0.26. 2026.

[41] Poly Haven. *Texturas y HDRI de dominio público (CC0)*. https://polyhaven.com. 2026.

[42] Ouster Inc. *OS1 Mid-Range High-Resolution Imaging Lidar: datasheet*. https://ouster.com/products/hardware/os1-lidar-sensor. 2023.

[43] Intel Corporation. *Intel RealSense Product Family D400 Series: datasheet*. `https://www.intelrealsense.com/depth-camera-d455/`. 2023.

[44] Rerun Technologies AB. *Rerun: an SDK for logging, storing and visualizing multimodal data*. `https://rerun.io`. Versión 0.41. 2026.